

Second Largest Element in an Array

Last Updated : 10 Feb, 2025

- Given an array of **positive** integers `arr[]` of size `n`, the task is to find **second largest distinct element** in the array.

Note: If the second largest element does not exist, return **-1**.

Examples:

Input: `arr[] = [12, 35, 1, 10, 34, 1]`

Output: 34

Explanation: The largest element of the array is 35 and the second largest element is 34.

Input: `arr[] = [10, 5, 10]`

Output: 5

Explanation: The largest element of the array is 10 and the second largest element is 5.

Input: `arr[] = [10, 10, 10]`

Output: -1

Explanation: The largest element of the array is 10 there is no second largest element.

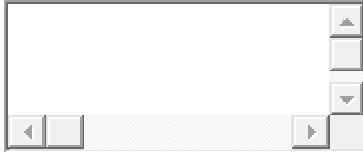
[Try It Yourself](#)

Table of Content

- [\[Naive Approach\] Using Sorting](#)
- [\[Better Approach\] Two Pass Search](#)
- [\[Expected Approach\] One Pass Search](#)

[Naive Approach] Using Sorting

The idea is to sort the array in **non-decreasing** order. Now, we know that the largest element will be at index `n - 1`. So, starting from index `(n - 2)`, traverse the remaining array in **reverse order**. As soon as we encounter an element which is **not equal** to the largest element, return it as the **second largest element** in the array. If all the elements are equal to the largest element, return **-1**.



```
# Python program to find second largest element in an array
# using Sorting

def getSecondLargest(arr):
    n = len(arr)

    # Sort the array in non-decreasing order
    arr.sort()

    # start from second last element as last element is the
    # largest
    for i in range(n - 2, -1, -1):

        # return the first element which is not equal to the
        # largest element
        if arr[i] != arr[n - 1]:
            return arr[i]

    # If no second largest element was found, return -1
    return -1

if __name__ == "__main__":
    arr = [12, 35, 1, 10, 34, 1]
    print(getSecondLargest(arr))
```

Output

34

Time Complexity: $O(n \cdot \log(n))$, as sorting the array takes $O(n \cdot \log(n))$ time and traversing the array can take $O(n)$ time in the worst case, so total time complexity = $(n \cdot \log(n) + n) = O(n \cdot \log(n))$.

Auxiliary space: $O(1)$, as no extra space is required.

[Better Approach] Two Pass Search

*The approach is to traverse the array **twice**. In the first traversal, find the **maximum** element. In the second traversal, find the maximum element **ignoring the one we found in the first traversal**.*

Working:



```
# Python program to find the second largest element in the array  
# using two traversals
```

```
# Function to find the second largest element in the array
```

```
def getSecondLargest(arr):  
    n = len(arr)  
  
    largest = -1  
    secondLargest = -1  
  
    # Finding the largest element  
    for i in range(n):  
        if arr[i] > largest:  
            largest = arr[i]  
  
    # Finding the second largest element  
    for i in range(n):  
  
        # Update second largest if the current element is  
        greater  
        # than second largest and not equal to the largest  
        if arr[i] > secondLargest and arr[i] != largest:  
            secondLargest = arr[i]  
  
    return secondLargest  
  
if __name__ == "__main__":  
    arr = [12, 35, 1, 10, 34, 1]  
    print(getSecondLargest(arr))
```

Output

34

Time Complexity: $O(2*n) = O(n)$, as we are traversing the array two times.

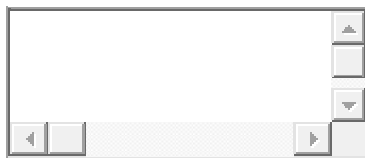
Auxiliary space: $O(1)$, as no extra space is required.

[Expected Approach] One Pass Search

The idea is to keep track of the **largest** and **second largest** element while traversing the array. Initialize largest and secondLargest with **-1**. Now, for any index i ,

- If $\text{arr}[i] > \text{largest}$, update secondLargest with largest and largest with $\text{arr}[i]$.
- Else If $\text{arr}[i] < \text{largest}$ and $\text{arr}[i] > \text{secondLargest}$, update secondLargest with $\text{arr}[i]$.

Working:



```
# Python program to find the second largest  
element in the array  
# using one traversal
```

```
# function to find the second largest element in the array  
def getSecondLargest(arr):  
    n = len(arr)  
  
    largest = -1  
    secondLargest = -1  
  
    # finding the second largest element  
    for i in range(n):  
  
        # If arr[i] > largest, update second largest with  
        # largest and largest with arr[i]  
        if arr[i] > largest:  
            secondLargest = largest  
            largest = arr[i]  
  
        # If arr[i] < largest and arr[i] > second largest,  
        # update second largest with arr[i]  
        elif arr[i] < largest and arr[i] > secondLargest:  
            secondLargest = arr[i]  
  
    return secondLargest  
  
if __name__ == "__main__":  
    arr = [12, 35, 1, 10, 34, 1]  
    print(getSecondLargest(arr))
```

Output

34

Time Complexity: $O(n)$, as we are traversing the array only once.

Auxiliary space: $O(1)$

2.

Third largest element in an array

Last Updated : 24 Apr, 2026

- Given an array of n integers, the task is to find the **third largest** element.

Examples :

Input: $arr[] = [2, 4, 1, 3, 5]$

Output: 3

Explanation: The third largest element in the array $[2, 4, 1, 3, 5]$ is 3.

Input: $arr[] = [10, 2]$

Output: -1

Explanation: There are less than three elements in the array, so the third largest element cannot be determined.

Input: $arr[] = [5, 5, 5]$

Output: 5

Explanation: In the array $[5, 5, 5]$, the third largest element can be considered 5, as there are no other distinct elements.

[Try It Yourself](#)

Table of Content

- [\[Naive Approach\] Using Sorting - \$O\(n * \log n\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Expected Approach - 1\] Using Three Loops - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Expected Approach - 2\] Using Single Loop - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)

[Naive Approach] Using Sorting - $O(n * \log n)$ Time and $O(1)$ Space

The idea is to sort the array and return the third largest element in the array which will be present at $(n-3)$ 'th index.



```
def thirdLargest(arr):  
    n = len(arr)  
  
    if n < 3:  
        return -1  
    # Sort the array  
    arr.sort()  
  
    # Return the third largest element  
    return arr[n - 3]  
  
if __name__ == "__main__":  
    arr = [2, 4, 1, 3, 5]  
    print(thirdLargest(arr))
```

Output

3

[Expected Approach - 1] Using Three Loops - $O(n)$ Time and $O(1)$ Space

The idea is to iterate the array twice and mark the maximum and second maximum element and then excluding them both find the third maximum element, i.e., the maximum element excluding the maximum and second maximum.

Step by step approach:

- First, iterate through the array and find maximum.
- Store this as first maximum along with its index.
- Now traverse the whole array finding the second max, excluding the maximum element.

- Finally traverse the array the third time and find the third largest element i.e., excluding the maximum and second maximum.



```
def thirdLargest(arr):
    n = len(arr)

    if n < 3:
        return -1

    # Find the first maximum element.
    first = -1
    first_idx = float('-inf')
    for i in range(n):
        if arr[i] > first:
            first = arr[i]
            first_idx = i

    # Find the second max element.
    second = -1
    second_idx = float('-inf')
    for i in range(n):
        if i == first_idx:
            continue

        if arr[i] > second:
            second = arr[i]
            second_idx = i

    # Find the third largest element.
    third = -1
    third = float('-inf')
    for i in range(n):
        if i == first_idx or i == second_idx:
            continue

        if arr[i] > third:
            third = arr[i]

    # Return the third largest element
    return third

if __name__ == "__main__":
    arr = [2, 4, 1, 3, 5]
    print(thirdLargest(arr))
```

Output

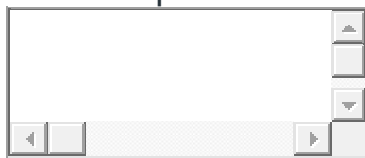
3

[Expected Approach - 2] Using Single Loop - $O(n)$ Time and $O(1)$ Space

The idea is to traverse the array from start to end and to keep track of the three largest elements up to that index (stored in variables). So after traversing the whole array, the variables would have stored the indices (or value) of the three largest elements of the array.

Step by step approach:

- Create three variables, first, second, third, to store indices of three largest elements of the array. (Initially all of them are initialized to a minimum value).
- Move along the input array from start to the end.
- For every index check if the element is larger than first or not. Update the value of first, if the element is larger, and assign the value of first to second and second to third. So the largest element gets updated and the elements previously stored as largest becomes second largest, and the second largest element becomes third largest.
- Else if the element is larger than the second, then update the value of second, and the second largest element becomes third largest.
- If the previous two conditions fail, but the element is larger than the third, then update the third.



```
def thirdLargest(arr):  
    n = len(arr)  
  
    first, second, third = float('-inf'), float('-inf'),  
float('-inf')  
  
    if n < 3:  
        return -1  
  
    for i in range(n):  
  
        # If arr[i] is greater than first,  
        # set third to second, second to
```



```

    # first and first to arr[i].
    if arr[i] > first:
        third = second
        second = first
        first = arr[i]

    # If arr[i] is greater than second,
    # set third to second and second
    # to arr[i].
    elif arr[i] > second:
        third = second
        second = arr[i]

    # If arr[i] is greater than third,
    # set third to arr[i].
    elif arr[i] > third:
        third = arr[i]

    # Return the third largest element
    return third

if __name__ == "__main__":
    arr = [2, 4, 1, 3, 5]
    print(thirdLargest(arr))

```

Output

3

3.

Array Reverse

Last Updated : 29 Mar, 2026

- Reverse an array **arr[]**. Reversing an array means **rearranging** the elements such that the **first** element becomes the **last**, the **second** element becomes **second last** and so on.

Examples:

Input: *arr[]* = [1, 4, 3, 2, 6, 5]

Output: [5, 6, 2, 3, 4, 1]

Explanation: The first element **1** moves to last position, the second element **4** moves to second-last and so on.

Input: `arr[] = [4, 5, 1, 2]`

Output: `[2, 1, 5, 4]`

Explanation: The first element **4** moves to last position, the second element **5** moves to second last and so on.

[Try It Yourself](#)

Table of Content

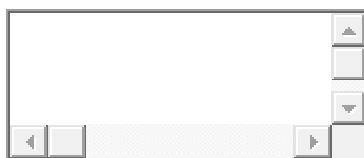
- [\[Naive Approach\] Using a temporary array - O\(n\) Time and O\(n\) Space](#)
- [\[Expected Approach - 1\] Using Two Pointers - O\(n\) Time and O\(1\) Space](#)
- [\[Expected Approach - 2\] Using Single Pointer - O\(n\) Time and O\(1\) Space](#)
- [Using Inbuilt Methods - O\(n\) Time and O\(1\) Space](#)

[Naive Approach] Using a temporary array - O(n) Time and O(n) Space

The idea is to use a **temporary array** to store the reverse of the array.

- Create a **temporary** array of same size as the original array.
- Now, copy all elements from original array to the temporary array in **reverse order**.
- Finally, copy all the elements from temporary array back to the original array.

Working:



```
def reverseArray(arr):  
    n = len(arr)  
  
    # Temporary array to store elements  
    # in reversed order  
    temp = [0] * n
```

```

# Copy elements from original array
# to temp in reverse order
for i in range(n):
    temp[i] = arr[n - i - 1]

# Copy elements back to original array
for i in range(n):
    arr[i] = temp[i]

if __name__ == "__main__":
    arr = [1, 4, 3, 2, 6, 5]

    reverseArray(arr)

    for i in range(len(arr)):
        print(arr[i], end=" ")

```

Output

```
5 6 2 3 4 1
```

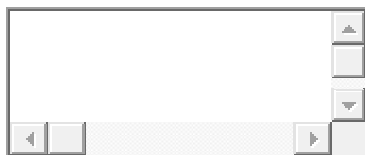
Time Complexity: $O(n)$, Copying elements to a new array is a linear operation.

Auxiliary Space: $O(n)$, as we are using an extra array to store the reversed array.

[Expected Approach - 1] Using Two Pointers - $O(n)$ Time and $O(1)$ Space

The idea is to maintain two pointers: **left** and **right**, such that **left** points at the **beginning** of the array and **right** points to the **end** of the array. While left pointer is less than the right pointer, swap the elements at these two positions. After each swap, **increment** the **left** pointer and **decrement** the **right** pointer to move towards the center of array. This will swap all the elements in the first half with their corresponding element in the second half.

Working:



```
def reverseArray(arr):
```

```

# Initialize left to the beginning
# and right to the end
left = 0
right = len(arr) - 1

# Iterate till left is less than right
while left < right:

    # Swap the elements at left
    # and right position
    arr[left], arr[right] = arr[right], arr[left]

    # Increment the left pointer
    left += 1

    # Decrement the right pointer
    right -= 1

if __name__ == "__main__":
    arr = [1, 4, 3, 2, 6, 5]

    reverseArray(arr)

    for i in range(len(arr)):
        print(arr[i], end=" ")

```

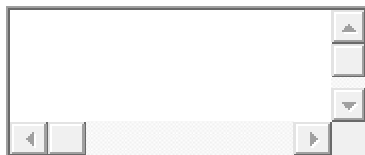
Output

```
5 6 2 3 4 1
```

[Expected Approach - 2] Using Single Pointer - O(n) Time and O(1) Space

The idea is to iterate over the **first half** of the array and **swap** each element with its corresponding element from the **end**. So, while iterating over the first half, any element at index **i** is swapped with the element at index **(n - i - 1)**.

Working:



```

def reverseArray(arr):
    n = len(arr)

```

```

# Iterate over the first half
# and for every index i, swap
# arr[i] with arr[n - i - 1]
for i in range(n // 2):
    temp = arr[i]
    arr[i] = arr[n - i - 1]
    arr[n - i - 1] = temp

if __name__ == "__main__":
    arr = [1, 4, 3, 2, 6, 5]

    reverseArray(arr)

    for i in range(len(arr)):
        print(arr[i], end=" ")

```

Output

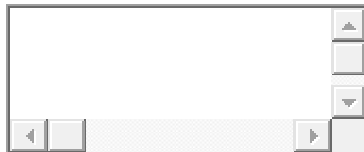
```
5 6 2 3 4 1
```

Time Complexity: $O(n)$, the loop runs through half of the array, so it's linear with respect to the array size.

Auxiliary Space: $O(1)$, no extra space is required, therefore we are reversing the array **in-place**.

Using Inbuilt Methods - $O(n)$ Time and $O(1)$ Space

*The idea is to use inbuilt **reverse** methods available across different languages.*



```

# function to reverse an array
def reverseArray(arr):
    arr.reverse()

if __name__ == "__main__":
    arr = [1, 4, 3, 2, 6, 5]

    reverseArray(arr)

    print(" ".join(map(str, arr)))

```

Output

```
5 6 2 3 4 1
```

Time Complexity: $O(n)$, the reverse method has linear time complexity.
Auxiliary Space: $O(1)$ Additional space is not used to store the reversed array, as the in-built array method swaps the values in-place.

4.

Reverse an Array in groups

Last Updated : 28 Mar, 2026

- Given an array `arr[]` and an integer `k`, find the array after reversing every subarray of consecutive `k` elements in place. If the last subarray has fewer than `k` elements, reverse it as it is. Modify the array in place, do not return anything.
Examples:
Input: `arr[] = [1, 2, 3, 4, 5, 6, 7, 8]`, `k = 3`
Output: `[3, 2, 1, 6, 5, 4, 8, 7]`
Explanation: Elements is reversed: `[1, 2, 3] → [3, 2, 1]`, `[4, 5, 6] → [6, 5, 4]`, and the last group `[7, 8]`(size < 3) is reversed as `[8, 7]`.
Input: `arr[] = [1, 2, 3, 4, 5]`, `k = 3`
Output: `[3, 2, 1, 5, 4]`
Explanation: First group consists of elements 1, 2, 3. Second group consists of 4, 5.
Input: `arr[] = [5, 6, 8, 9]`, `k = 5`
Output: `[9, 8, 6, 5]`
Explanation: Since `k` is greater than array size, the entire array is reversed.

[Try It Yourself](#)

Fixed-Size Group Reversal–Time $O(n)$ and Space $O(1)$

Edge Cases:

- When `k = 1`, the array stays the same
- When `k` is greater than or equal to the array size

Approach

- We begin from index 0 and find the size of the current subarray to be reversed. If the number of remaining elements at the end is less than k , reverse all of them.
- Each subarray is reversed using two pointers that start from the two corners of the subarray.



```
def reverseInGroups(arr, k):
    i = 0
    n = len(arr)

    while i < n:
        left = i

        # To handle case when k is not multiple of n
        right = min(i + k - 1, n - 1)

        # reverse the sub-array [left, right]
        while left < right:
            arr[left], arr[right] = arr[right], arr[left]
            left += 1
            right -= 1

        i += k

if __name__ == "__main__":
    arr = [1, 2, 3, 4, 5, 6, 7, 8]
    k = 3
    reverseInGroups(arr, k)
    print(" ".join(map(str, arr)))
```

Output

```
3 2 1 6 5 4 8 7
```

5.

Rotate an Array by d - Counterclockwise or Left

Last Updated : 23 Jul, 2025

- Given an array of integers **arr[]** of size **n**, the task is to **rotate** the array elements to the **left** by **d** positions.

Examples:

Input: $arr[] = \{1, 2, 3, 4, 5, 6\}$, $d = 2$

Output: $\{3, 4, 5, 6, 1, 2\}$

Explanation: After first left rotation, $arr[]$ becomes $\{2, 3, 4, 5, 6, 1\}$ and after the second rotation, $arr[]$ becomes $\{3, 4, 5, 6, 1, 2\}$

Input: $arr[] = \{1, 2, 3\}$, $d = 4$

Output: $\{2, 3, 1\}$

Explanation: The array is rotated as follows:

- After first left rotation, $arr[] = \{2, 3, 1\}$
- After second left rotation, $arr[] = \{3, 1, 2\}$
- After third left rotation, $arr[] = \{1, 2, 3\}$
- After fourth left rotation, $arr[] = \{2, 3, 1\}$

[Try It Yourself](#)

Table of Content

- [\[Naive Approach\] Rotate one by one - \$O\(n * d\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Better Approach\] Using Temporary Array - \$O\(n\)\$ Time and \$O\(n\)\$ Space](#)
- [\[Expected Approach 1\] Using Juggling Algorithm - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Expected Approach 2\] Using Reversal Algorithm - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)

[Naive Approach] Rotate one by one - $O(n * d)$ Time and $O(1)$ Space

In each iteration, shift the elements by one position to the left in a **circular** fashion (the first element becomes the last). Perform this operation **d** times to rotate the elements to the left by d positions.

Illustration:

Let us take $arr[] = \{1, 2, 3, 4, 5, 6\}$, $d = 2$.

First Step:

=> Rotate to left by one position.

=> $arr[] = \{2, 3, 4, 5, 6, 1\}$

Second Step:

=> Rotate again to left by one position

=> $arr[] = \{3, 4, 5, 6, 1, 2\}$

Rotation is done 2 times.

So the array becomes $arr[] = \{3, 4, 5, 6, 1, 2\}$



Python Program to left rotate the array by d positions
by rotating one element at a time

Function to left rotate array by d positions

```
def rotateArr(arr, d):  
    n = len(arr)
```

```
    # Repeat the rotation d times  
    for i in range(d):
```

```
        # Left rotate the array by one position  
        first = arr[0]  
        for j in range(n - 1):  
            arr[j] = arr[j + 1]  
        arr[n - 1] = first
```

```
if __name__ == "__main__":  
    arr = [1, 2, 3, 4, 5, 6]  
    d = 2  
  
    rotateArr(arr, d)  
  
    for i in range(len(arr)):  
        print(arr[i], end=" ")
```

Output

3 4 5 6 1 2

Time Complexity: $O(n*d)$, the outer loop runs d times, and within each iteration, the inner loop shifts all n elements of the array by one position, resulting in a total of $n*d$ operations.

Auxiliary Space: $O(1)$

[Better Approach] Using Temporary Array - $O(n)$ Time and $O(n)$ Space

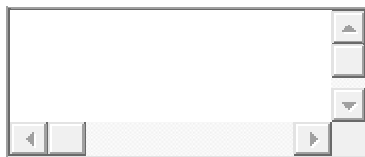
This problem can be solved using the below idea:

The idea is to use a temporary array of size n , where n is the length of the original array. If we left rotate the array by d positions, the last $n - d$ elements will be at the front and the first d elements will be at the end.

- Copy the **last $(n - d)$ elements** of original array into the first $n - d$ positions of temporary array.
- Then copy the **first d elements** of the original array to the end of temporary array.
- Finally, **copy all the elements** of temporary array back into the original array.

Working:

Below is the implementation of the algorithm:



```
# Python Program to left rotate the array by d positions  
# using temporary array
```

```
# Function to rotate array
```

```
def rotateArr(arr, d):
```

```
    n = len(arr)
```

```
    # Handle case when d > n
```

```
    d %= n
```

```
    # Storing rotated version of array
```

```
    temp = [0] * n
```

```
    # Copy last n - d elements to the front of temp
```

```
    for i in range(n - d):
```

```
        temp[i] = arr[d + i]
```

```
    # Copy the first d elements to the back of temp
```

```
    for i in range(d):
```

```
        temp[n - d + i] = arr[i]
```

```
    # Copying the elements of temp in arr
```

```
    # to get the final rotated array
```

```
    for i in range(n):
```

```

        arr[i] = temp[i]

if __name__ == "__main__":
    arr = [1, 2, 3, 4, 5, 6]
    d = 2

    rotateArr(arr, d)

    # Print the rotated array
    for i in range(len(arr)):
        print(arr[i], end=" ")

```

Output

```
3 4 5 6 1 2
```

Time Complexity: $O(n)$, as we are visiting each element only twice.

Auxiliary Space: $O(n)$, as we are using an additional temporary array.

[Expected Approach 1] Using Juggling Algorithm - $O(n)$ Time and $O(1)$ Space

The idea is to use Juggling Algorithm which is based on rotating elements in **cycles**. Each cycle is **independent** and represents a group of elements that will shift among themselves during the rotation. If the starting index of a cycle is i , then next elements of the cycle can be found at indices $(i + d) \% n$, $(i + 2d) \% n$, $(i + 3d) \% n$... and so on till we return to the original index i .

So for any index i , we know that after rotation, the element that will occupy this position is $arr[(i + d) \% n]$. Consequently, for every index in the cycle, we will place the element that should be in that position after the rotation is completed.

Please refer [Juggling Algorithm for Array Rotation](#) to know more about the implementation.

Time Complexity: $O(n)$

Auxiliary Space: $O(1)$

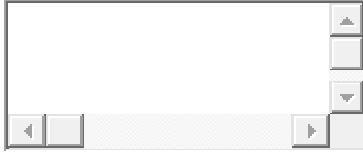
[Expected Approach 2] Using Reversal Algorithm - $O(n)$ Time and $O(1)$ Space

The idea is based on the observation that if we left rotate the array by d positions, the last $(n - d)$ elements will be at the front and the first d elements will be at the end.

- **Reverse** the subarray containing the **first d** elements of the array.
- **Reverse** the subarray containing the **last $(n - d)$** elements of the array.
- Finally, **reverse all the elements** of the array.

Working:

Below is the implementation of the algorithm:



```
# Python Code to left rotate an array using Reversal Algorithm
```

```
# Function to rotate an array by d elements to the left
```

```
def rotateArr(arr, d):  
    n = len(arr)
```

```
    # Handle the case where d > size of array  
    d %= n
```

```
    # Reverse the first d elements  
    reverse(arr, 0, d - 1)
```

```
    # Reverse the remaining n-d elements  
    reverse(arr, d, n - 1)
```

```
    # Reverse the entire array  
    reverse(arr, 0, n - 1)
```

```
# Function to reverse a portion of the array
```

```
def reverse(arr, start, end):  
    while start < end:  
        arr[start], arr[end] = arr[end], arr[start]  
        start += 1  
        end -= 1
```

```
if __name__ == "__main__":  
    arr = [1, 2, 3, 4, 5, 6]  
    d = 2  
  
    rotateArr(arr, d)  
  
    for i in range(len(arr)):  
        print(arr[i], end=" ")
```

Output

```
3 4 5 6 1 2
```

Time complexity: $O(n)$, as we are visiting each element exactly twice.
Auxiliary Space: $O(1)$

6.

Maximum product of a triplet (subsequence of size 3) in array

Last Updated : 12 Feb, 2025

- Given an integer array, find a maximum product of a triplet in the array.

Examples:

Input: `arr[] = [10, 3, 5, 6, 20]`

Output: 1200

Explanation: Multiplication of 10, 6 and 20

Input: `arr[] = [-10, -3, -5, -6, -20]`

Output: -90

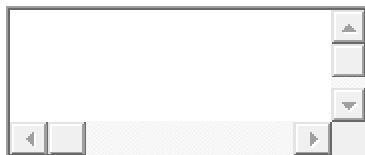
Input: `arr[] = [1, -4, 3, -6, 7, 0]`

Output: 168

[Try It Yourself](#)

[Naive Approach] By Using three nested loops

A simple solution is to check for every triplet using three nested loops.



```
# Python program to find a maximum product of a
# triplet in array of integers using nested loops

# Function to find a maximum product of a triplet
# in array of integers of size n
def maxProduct(arr):
    n = len(arr)
```

```

maxProduct = -10**9

for i in range(n - 2):
    for j in range(i + 1, n - 1):
        for k in range(j + 1, n):
            maxProduct = max(maxProduct, arr[i] * arr[j] *
arr[k])

return maxProduct

if __name__ == "__main__":
    arr = [10, 3, 5, 6, 20]
    print(maxProduct(arr))

```

Output

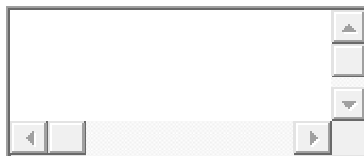
1200

Time Complexity: $O(n^3)$

Auxiliary Space: $O(1)$

[Better Approach] By Using sorting - Time $O(n \cdot \log(n))$ and Space $O(1)$

- Sort the array using some efficient in-place sorting algorithm in ascending order.
- In triplets, either there will be 2 negative elements and 1 positive element or all 3 positive elements so that resultant product will be positive.
- Therefore, To maximise the result return the maximum of product of the last three elements of the array and the product of the first two elements and last element.



```

# A Python3 program to find a maximum
# product of a triplet in an array of integers
# using sorting

# Function to find a maximum product of a triplet
# in array of integers of size n
def maxProduct(arr):

```

```

n = len(arr)

# Sort the array in ascending order
arr.sort()

# Return the maximum of product of last three
# elements and product of first two elements
# and last element
return max(arr[0] * arr[1] * arr[n - 1],
           arr[n - 1] * arr[n - 2] * arr[n - 3])

if __name__ == "__main__":
    arr = [-10, -3, 5, 6, -20]
    max = maxProduct(arr)
    print(max)

```

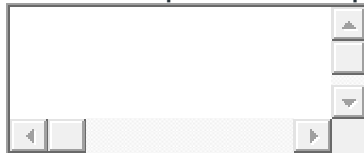
Output

1200

[Expected Approach] By Using Greedy approach - Time $O(n)$ and Space $O(1)$

- Scan the array and compute the Maximum, second maximum and third maximum element present in the array.
- Scan the array and compute Minimum and second minimum element present in the array.
- Return the maximum of product of Maximum, second maximum and third maximum and product of Minimum, second minimum and Maximum element.

Note: Step 1 and Step 2 can be done in a single traversal of the array.



```

# Function to find a maximum product of a triplet
# in array of integers of size n
def maxProduct(arr):
    n = len(arr)

    # Initialize Maximum, second maximum
    # and third maximum element
    maxA = float('-inf')
    maxB = float('-inf')

```

```

maxC = float('-inf')

# Initialize Minimum and second minimum element
minA = float('inf')
minB = float('inf')

for i in range(n):

    # Update Maximum, second maximum
    #and third maximum element
    if arr[i] > maxA:
        maxC = maxB
        maxB = maxA
        maxA = arr[i]
    elif arr[i] > maxB:
        maxC = maxB
        maxB = arr[i]
    elif arr[i] > maxC:
        maxC = arr[i]

    # Update Minimum and second minimum element
    if arr[i] < minA:
        minB = minA
        minA = arr[i]
    elif arr[i] < minB:
        minB = arr[i]

    return max(minA * minB * maxA, maxA * maxB * maxC)

if __name__ == "__main__":
    arr = [-10, -3, 5, 6, -20]
    print(maxProduct(arr))

```

Output

```
1200
```

7.

Maximum consecutive one's (or zeros) in a binary array

Last Updated : 25 Jun, 2025

- Given a binary array **arr[]** consisting of only **0s** and **1s**, find the length of the **longest contiguous sequence** of either 1s or 0s in the array.

Examples :

Input: `arr[] = [0, 1, 0, 1, 1, 1, 1]`

Output: 4

Explanation: The maximum number of consecutive 1's in the array is 4 from index 3-6.

Input: `arr[] = [0, 0, 1, 0, 1, 0]`

Output: 2

Explanation: The maximum number of consecutive 0's in the array is 2 from index 0-1.

Input: `arr[] = [0, 0, 0, 0]`

Output: 4

Explanation: The maximum number of consecutive 0's in the array is 4.

[Try It Yourself](#)

Table of Content

- [\[Approach - 1\] Using Simple Traversal - O\(n\) Time and O\(1\) Space](#)
- [\[Approach - 2\] Using Bit Manipulation - O\(n\) Time and O\(1\) Space](#)

[Approach - 1] Using Simple Traversal - O(n) Time and O(1) Space

The idea is to **traverse** the array once and count how many times the **same value repeats consecutively**. The thought process is to use a **counter** that increases when the current element matches the previous one. If a change is detected, we **update** the **maximum streak** and reset the count.

Steps-By-Step Approach:

- Initialize two variables: **maxCount = 0** to store result and **count = 1** for current streak length.
- Iterate** through the array starting from index 1 to compare each element with the previous one.
- If **arr[i] equals arr[i - 1]**, it means the streak continues, so increment **count** by 1.
- If they differ, **update maxCount** with the current **count** and reset **count = 1** for a new streak.

- Continue this comparison until the loop finishes checking all elements in the array.
- After the loop, take the **maximum of maxCount and count** to cover the last streak.
- **Return maxCount** which holds the maximum number of consecutive 1s or 0s in the array.



```
def maxConsecBits(arr):
    maxCount, count = 0, 1

    # Loop through the array starting from the second element
    for i in range(1, len(arr)):

        # If the current element is the same as the previous one
        # increment the count
        if arr[i] == arr[i - 1]:
            count += 1
        # If not equal, update maxCount if needed and reset
        count
    else:
        maxCount = max(maxCount, count)
        count = 1

    return max(maxCount, count)

if __name__ == "__main__":
    arr = [0, 1, 0, 1, 1, 1, 1]
    print(maxConsecBits(arr))
```

Output

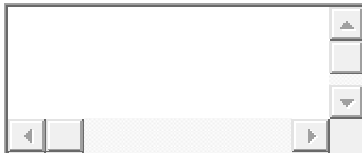
4

[Approach - 2] Using Bit Manipulation - O(n) Time and O(1) Space

The **idea** is to use **XOR** (^) to check if **consecutive** elements are the same. As **XOR** of two numbers is **0** if both numbers are same. So, If **prev ^ num == 0**, the sequence continues; else we update the **maximum streak** and **reset** the count.

Steps to implement the above idea:

- Initialize three variables: **maxCount = 0**, **count = 0**, and **prev = -1** to track current and previous bits.
- Traverse the array, use **XOR** to compare current **num** with **prev**, if equal, increment **count** for the current streak.
- If **prev and num differ**, update **maxCount** with the current **count**, then reset **count to 1**.
- Update **prev** to current **num** in every iteration to track the previous element for the next comparison.
- After loop ends, compare and return the **maximum of maxCount and count** to cover last streak.



```
def maxConsecBits(arr):
    maxCount, count, prev = 0, 0, -1

    for num in arr:

        # If the current number is the same
        # as the previous number
        if (prev ^ num) == 0:
            count += 1
        else:

            # Update max_count and reset count
            maxCount = max(maxCount, count)
            count = 1
            prev = num

    return max(maxCount, count)

if __name__ == "__main__":
    arr = [0, 1, 0, 1, 1, 1, 1]

    print(maxConsecBits(arr))
```

Output

4

Move all Zeros to End of Array

Last Updated : 26 Feb, 2026

- Given an array of integers **arr[]**, move all the **zeros** to the **end** of the array while maintaining the relative order of all **non-zero** elements.

Examples:

Input: *arr[] = [1, 2, 0, 4, 3, 0, 5, 0]*

Output: *[1, 2, 4, 3, 5, 0, 0, 0]*

Explanation: *There are three 0s that are moved to the end.*

Input: *arr[] = [10, 20, 30]*

Output: *[10, 20, 30]*

Explanation: *No change in array as there are no 0s.*

Input: *arr[] = [0, 0]*

Output: *[0, 0]*

Explanation: *No change in array as there are all 0s.*

[Try It Yourself](#)

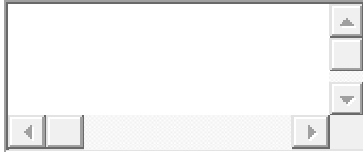
Table of Content

- [\[Naive Approach\] Using Temporary Array - O\(n\) Time and O\(n\) Space](#)
- [\[Better Approach\] Two Traversals-O\(n\) Time and O\(1\) space](#)
- [\[Expected Approach\] One Traversal-O\(n\) Time and O\(1\) space](#)

[Naive Approach] Using Temporary Array - O(n) Time and O(n) Space

The idea is to use a temporary array of the same size, copy all non-zero elements into it, fill the remaining positions with zeros, and then copy the temporary array back to the original array.

Working:



```
def pushZerosToEnd(arr):  
    n = len(arr)  
    temp = [0] * n  
  
    # to keep track of the index in temp[]  
    j = 0  
  
    # Copy non-zero elements to temp[]  
    for i in range(n):  
        if arr[i] != 0:  
            temp[j] = arr[i]  
            j += 1  
  
    # Fill remaining positions in temp[] with zeros  
    while j < n:  
        temp[j] = 0  
        j += 1  
  
    # Copy all the elements from temp[] to arr[]  
    for i in range(n):  
        arr[i] = temp[i]  
  
if __name__ == "__main__":  
    arr = [1, 2, 0, 4, 3, 0, 5, 0]  
    pushZerosToEnd(arr)  
  
    for num in arr:  
        print(num, end=" ")
```

Output

```
1 2 4 3 5 0 0 0
```

[Better Approach] Two Traversals- $O(n)$ Time and $O(1)$ space

The idea is to move all the zeros to the end of the array while maintaining the relative order of non-zero elements using two traversals. Traverse the array once to move all non-zero elements to the front while maintaining order, then traverse the remaining positions and fill them with zeros.

Working:



```
def pushZerosToEnd(arr):  
  
    count = 0  
  
    # If the element is non-zero, replace the element at  
    # index 'count' with this element and increment count  
    for i in range(len(arr)):  
        if arr[i] != 0:  
            arr[count] = arr[i]  
            count += 1  
  
    # Now all non-zero elements have been shifted to  
    # the front. Make all elements 0 from count to end.  
    while count < len(arr):  
        arr[count] = 0  
        count += 1  
  
if __name__ == "__main__":  
    arr = [1, 2, 0, 4, 3, 0, 5, 0]  
    pushZerosToEnd(arr)  
    for num in arr:  
        print(num, end=" ")
```

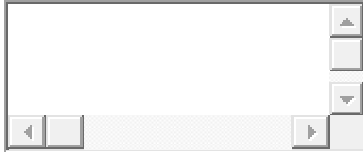
Output

```
1 2 4 3 5 0 0 0
```

[Expected Approach] One Traversal- $O(n)$ Time and $O(1)$ space

The idea is similar to the previous approach where we took a pointer, say *count* to track where the next non-zero element should be placed. However, on encountering a non-zero element, instead of directly placing the non-zero element at *arr[count]*, we will swap the non-zero element with *arr[count]*. This will ensure that if there is any zero present at *arr[count]*, it is pushed towards the end of array and is not overwritten.

Working:



```
def pushZerosToEnd(arr):  
    # Pointer to track the position  
    # for next non-zero element  
    count = 0  
  
    for i in range(len(arr)):  
        # If the current element is non-zero  
        if arr[i] != 0:  
            # Swap the current element with  
            # the 0 at index 'count'  
            arr[i], arr[count] = arr[count], arr[i]  
  
            # Move 'count' pointer to the next position  
            count += 1  
  
if __name__ == "__main__":  
    arr = [1, 2, 0, 4, 3, 0, 5, 0]  
    pushZerosToEnd(arr)  
    for num in arr:  
        print(num, end=" ")
```

Output

```
1 2 4 3 5 0 0 0
```

9.

Sort an array in wave form

Last Updated : 22 Jul, 2025

- Given a sorted array **arr[]** of integers (in ascending order), rearrange the elements in-place to form a wave-like array.
An array is said to be in wave form if it satisfies the following pattern: **arr[0] ≥ arr[1] ≤ arr[2] ≥ arr[3] ≤ arr[4] ≥ ...**

In other words, every even-indexed element should be greater than or equal to its adjacent odd-indexed elements (if they exist).

Note: The given array is sorted in ascending order, and modify the given array in-place without returning a new array.

Input: `arr[] = [1, 2, 3, 4, 5]`

Output: `[2, 1, 4, 3, 5]`

Explanation: Array elements after sorting it in the waveform are 2, 1, 4, 3, 5.

Input: `arr[] = [2, 4, 7, 8, 9, 10]`

Output: `[4, 2, 8, 7, 10, 9]`

Explanation: Array elements after sorting it in the waveform are 4, 2, 8, 7, 10, 9.

Try It Yourself

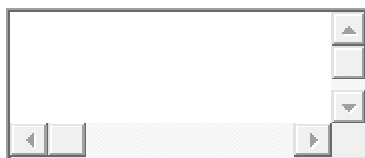
[Approach] Adjacent Pair Swapping Method

The main idea of this approach is to rearrange it into a wave form by swapping adjacent elements in pairs.

Since, the elements are in increasing order. Then, by swapping every pair of adjacent elements (i.e., `arr[0]` with `arr[1]`, `arr[2]` with `arr[3]`, and so on), we create a pattern where:

- Every even index i holds a value greater than or equal to its neighboring odd indices $i - 1$ and $i + 1$ (if they exist).
- This guarantees the wave condition: $arr[0] \geq arr[1] \leq arr[2] \geq arr[3] \leq arr[4] \geq \dots$

The sorted array ensures the numbers are close enough to form valid wave peaks and valleys, and the pairwise swaps are what enforce the pattern.



```
def sortInWave(arr):  
  
    n = len(arr)  
  
    # swap adjacent elements  
    for i in range(0, n-1, 2):  
        arr[i], arr[i+1] = arr[i+1], arr[i]
```



```

if __name__ == "__main__":
    arr = [1, 2, 3, 4, 5]
    sortInWave(arr)
    for i in range(0, len(arr)):
        print (arr[i], end=" ")

```

Output

```
2 1 4 3 5
```

Time Complexity: $O(n)$, a single pass swaps adjacent pairs.

Auxiliary Space: $O(1)$, in-place swaps use no extra storage.

10.

Adding one to number represented as array of digits

Last Updated : 31 Oct, 2025

- Given a non-negative number represented as an array of digits. The task is to add 1 to the number (increment the number represented by the digits by 1). The digits are stored such that the most significant digit is the first element of the array.

Examples :

Input : [1, 2, 4]

Output : 125

Explanation: $124 + 1 = 125$

Input : [9, 9, 9]

Output: 1000

Explanation: $999 + 1 = 1000$

[Try It Yourself](#)

Table of Content

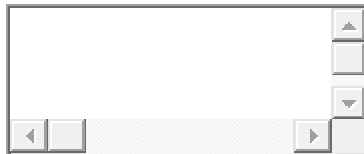
- [\[Approach - 1\] - Using Carry - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Approach - 2\] - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)

- [Alternate Approach] - $O(n)$ Time and $O(1)$ Space

[Approach - 1] - Using Carry - $O(n)$ Time and $O(1)$ Space

To add one to the number represented by digits, follow the below steps :

- Parse the given array from the end as we do in school addition.
- If the last elements are 9, make it 0 and carry = 1.
- For the next iteration check carry and if it adds to 10, do the same as step 2.
- After adding carry, make carry = 0 for the next iteration.
- If the carry still remains after traversing the entire array, append 1 in the beginning.



```
# Python program to add 1 to a  
# number represented as an array
```

```
def addOne(arr):  
  
    carry = 1  
  
    for i in range(len(arr) - 1, -1, -1):  
        sum = arr[i] + carry  
        arr[i] = sum % 10  
        carry = sum // 10  
  
    if carry:  
        arr.insert(0, carry)  
  
    return arr  
  
# Driver code  
if __name__ == "__main__":  
    arr = [9, 9, 9]  
    res = addOne(arr)  
    for i in res:  
        print(i, end="")
```

Output

```
1000
```

Time Complexity: $O(n)$, where n is the size of array.

Space Complexity: $O(1)$

[Approach - 2] - $O(n)$ Time and $O(1)$ Space

The idea is to start from the end of the vector and if the element is 9 set it to 0, else increment the digit by 1 and exit the loop.

- *If the loop set all digits to 0 (if all digits were 9) insert 1 at the beginning.*
- *Else increment the element at the position where the loop stopped.*



```
# Python program to add 1 to a
# number represented as an array

# function to add one
def addOne(arr):

    # initialize an index to end of array
    index = len(arr) - 1

    # while the index is valid and the value
    # at index is 9
    while index >= 0 and arr[index] == 9:
        arr[index] = 0
        index -= 1

    # if index < 0 (if all arr were 9)
    if index < 0:

        # insert an one at the beginning of the list
        arr.insert(0, 1)

    # else increment the value at [index]
    else:
        arr[index] += 1

    return arr

if __name__ == "__main__":

    # Driver code
    arr = [9, 9, 9]
```

```
res = addOne(arr)
for i in res:
    print(i, end="")
```

Output

1000

Time Complexity: $O(n)$, where n is the number of digits / size of the array.

Auxiliary Space: $O(1)$

[Alternate Approach] - $O(n)$ Time and $O(1)$ Space

To avoid inserting at front of array, we can firstly reverse the array, and then append the value at the last. Thereafter, we can reverse the array again to get the result.



```
# Python program to add 1 to a
# number represented as an array

# function to add one
def addOne(arr):

    # reverse the digits
    arr.reverse()

    # initialize an index to start of array
    index = 0

    # while the index is valid and the value
    # at index is 9
    while index < len(arr) and arr[index] == 9:
        arr[index] = 0
        index += 1

    # if index == len(arr) (if all arr were 9)
    if index == len(arr):

        # insert an one at the end
        arr.append(1)

    # else increment the value at [index]
    else:
```

```

        arr[index] += 1

    # reverse the array
    arr.reverse()

    return arr

# Driver code
if __name__ == "__main__":
    arr = [9, 9, 9]
    res = addOne(arr)
    for i in res:
        print(i, end="")

```

Output

1000

Time Complexity: $O(n)$, where n is the number of digits / size of the array.
Auxiliary Space: $O(1)$

11.

Stock Buy and Sell - Max one Transaction Allowed

Last Updated : 31 Jan, 2026

- Given an array **prices[]** of non-negative integers, representing the prices of the stocks on different days, find the maximum profit possible by buying and selling the stocks on different days when **at most one** transaction is allowed. Here one transaction means 1 buy + 1 Sell. If it is not possible to make a profit then **return 0**.

Note: Stock must be bought before being sold.

Examples:

Input: `prices[] = [7, 10, 1, 3, 6, 9, 2]`

Output: 8

Explanation: Buy for price 1 and sell for price 9.

Input: `prices[] = [7, 6, 4, 3, 1]`

Output: 0

Explanation: Since the array is sorted in decreasing order, 0 profit can be made without making any transaction.

Input: `prices[] = [1, 3, 6, 9, 11]`

Output: 10

Explanation: Since the array is sorted in increasing order, we can make maximum profit by buying at price[0] and selling at price[n-1]

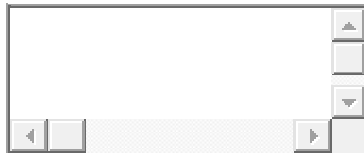
[Try It Yourself](#)

Table of Content

- [\[Naive Approach\] By exploring all possible pairs - O\(n²\) Time and O\(1\) Space](#)
- [\[Expected Approach\] One Traversal Solution - O\(n\) Time and O\(1\) Space](#)

[Naive Approach] By exploring all possible pairs - O(n²) Time and O(1) Space

The idea is to use two nested loops to explore all the possible ways to buy and sell stock. The outer loop decides the day to buy the stock and the inner loop decides the day to sell the stock. The maximum difference between the selling price and buying price between every pair of days will be our answer.



```
def max_profit(prices):
    n = len(prices)
    res = 0

    # Explore all possible ways to buy and sell stock
    for i in range(n - 1):
        for j in range(i + 1, n):
            res = max(res, prices[j] - prices[i])

    return res

if __name__ == "__main__":
    prices = [7, 10, 1, 3, 6, 9, 2]
    print(max_profit(prices))
```

Output

[Expected Approach] One Traversal Solution - $O(n)$ Time and $O(1)$ Space

In order to maximize the profit, we need to minimize the cost price and maximize the selling price. So at every step, we keep track of the minimum buy price of stock encountered so far. For every price, we subtract with the minimum so far and if we get more profit than the current result, we update the result.



```
def maxProfit(prices):
    minSoFar = prices[0]
    res = 0

    for i in range(1, len(prices)):

        # Update the minimum value seen so far
        minSoFar = min(minSoFar, prices[i])

        # Update result if we get more profit
        res = max(res, prices[i] - minSoFar)

    return res

if __name__ == "__main__":
    prices = [7, 10, 1, 3, 6, 9, 2]
    print(maxProfit(prices))
```

Output

12.

Stock Buy and Sell - Multiple Transaction Allowed

Last Updated : 9 Feb, 2026

- Given an array **prices[]** representing stock prices, find the maximum total profit that can be earned by buying and selling the stock any number of times.

Note: We can only sell a stock which we have bought earlier and we cannot hold multiple stocks on any day.

Examples:

Input: `prices[] = [100, 180, 260, 310, 40, 535, 695]`

Output: 865

Explanation: Buy the stock on day 0 and sell it on day 3 = $310 - 100 = 210$ and Buy the stock on day 4 and sell it on day 6 = $695 - 40 = 655$ so the Maximum Profit is = $210 + 655 = 865$.

Input: `prices[] = [4, 2]`

Output: 0

Explanation: Stock prices keep decreasing, there is no chance to sell at a higher price after buying, so no profit can be made.

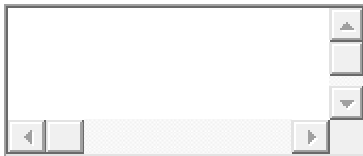
[Try It Yourself](#)

Table of Content

- [\[Naive Approach\] By Trying All Possibility - \$O\(2^n\)\$ Time and \$O\(n\)\$ Space](#)
- [\[Better Approach\] Using Local Minima and Maxima - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Expected Approach\] By Accumulating Profit - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)

[Naive Approach] By Trying All Possibility - $O(2^n)$ Time and $O(n)$ Space

The idea is to use recursion to simulate all choices of buying and selling. For each day, you can either skip it or buy on that day. If you buy at day i , then you try all possible selling days $j > i$ where $price[j] > price[i]$.



```
def maxProfitRec(price, start, end):
    res = 0

    # Try every possible pair of buy (i) and sell (j)
    for i in range(start, end):
        for j in range(i + 1, end + 1):

            # Valid transaction if selling price > buying price
            if price[j] > price[i]:
                curr = (price[j] - price[i]) + \
                    maxProfitRec(price, start, i - 1) + \
                    maxProfitRec(price, j + 1, end)
                res = max(res, curr)

    return res

def maxProfit(prices):
    return maxProfitRec(prices, 0, len(prices) - 1)

if __name__ == "__main__":
    prices = [100, 180, 260, 310, 40, 535, 695]
    print(maxProfit(prices))
```

Output

865

[Better Approach] Using Local Minima and Maxima - $O(n)$ Time and $O(1)$ Space

The idea is to traverse the array from left to right and Find local minima (where price starts rising) and then a local maxima (where price stops rising). Compute the difference between two and add to the result.

Below is the implementation of the algorithm



```
def maxProfit(prices):
    n = len(prices)
    lMin = prices[0]
    lMax = prices[0]
    res = 0

    i = 0
    while i < n - 1:

        # Find local minima
        while i < n - 1 and prices[i] >= prices[i + 1]:
            i += 1
        lMin = prices[i]

        # Local Maxima
        while i < n - 1 and prices[i] <= prices[i + 1]:
            i += 1
        lMax = prices[i]

        # Add current profit
        res += (lMax - lMin)

    return res

if __name__ == "__main__":
    prices = [100, 180, 260, 310, 40, 535, 695]
    print(maxProfit(prices))
```

Output

865

[Expected Approach] By Accumulating Profit - $O(n)$ Time and $O(1)$ Space

The idea is that profit only comes when prices rise. If the price goes up from one day to the next, we can think of it as buying yesterday and selling today. Instead of waiting for the exact bottom and top, we simply grab every small upward move. Adding these small gains together is the same as if we had bought at each valley and sold at each peak because every rise between them gets counted.

Below is the implementation of the algorithm:



```
def maxProfit(prices):  
    res = 0  
  
    # Keep on adding the difference between  
    # adjacent when the prices a  
    for i in range(1, len(prices)):  
        if prices[i] > prices[i - 1]:  
            res += prices[i] - prices[i - 1]  
  
    return res  
  
if __name__ == "__main__":  
    prices = [100, 180, 260, 310, 40, 535, 695]  
    print(maxProfit(prices))
```

Output

865

13.

Remove duplicates from Sorted Array

Last Updated : 19 Nov, 2024

- Given a sorted array `arr[]` of size `n`, the goal is to rearrange the array so that all distinct elements appear at the beginning in sorted order. Additionally, return the length of this distinct sorted subarray.

Note: The elements after the distinct ones can be in any order and hold any value, as they don't affect the result.

Examples:

Input: `arr[] = [2, 2, 2, 2, 2]`

Output: `[2]`

Explanation: All the elements are 2, So only keep one instance of 2.

Input: `arr[] = [1, 2, 2, 3, 4, 4, 4, 5, 5]`

Output: `[1, 2, 3, 4, 5]`

Input: `arr[] = [1, 2, 3]`

Output: `[1, 2, 3]`

Explanation : No change as all elements are distinct.

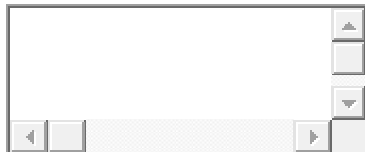
[Try It Yourself](#)

Table of Content

- [Using Hash Set - Works for Unsorted Also - O\(n\) Time and O\(n\) Space](#)
- [Expected Approach - O\(n\) Time and O\(1\) Space](#)

Using Hash Set - Works for Unsorted Also - O(n) Time and O(n) Space

- Use a hash set or dictionary to store elements already processed
- Initialize index of result array as 0.
- Traverse through the input array. If an element is not in the hash set, put it at the result index and insert into the set.



```
def removeDuplicates(arr):  
  
    # To track seen elements  
    seen = set()
```

```

# To maintain the new size of the array
idx = 0

for i in range(len(arr)):
    if arr[i] not in seen:
        seen.add(arr[i])
        arr[idx] = arr[i]
        idx += 1

# Return the size of the array
# with unique elements
return idx

if __name__ == "__main__":
    arr = [1, 2, 2, 3, 4, 4, 4, 5, 5]
    newSize = removeDuplicates(arr)

    for i in range(newSize):
        print(arr[i], end=" ")

```

Output

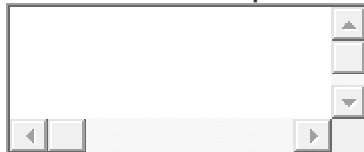
```
1 2 3 4 5
```

Expected Approach - $O(n)$ Time and $O(1)$ Space

Since the array is **sorted**, we do not need to maintain a **hash set**. All occurrences of an element would be consecutive. So we mainly need to check if the current element is same as the previous element or not.

Step by step implementation:

- Start with `idx = 1` (`idx` is going to hold the index of the next distinct item. Since there is nothing before the first item, we consider it as the first distinct item and begin `idx` with 1).
- Loop through the array for `i = 0` to `n-1`.
 - At each index `i`, if `arr[i]` is different from `arr[i-1]`, assign `arr[idx] = arr[i]` and increment `idx`.
- After the loop, `arr[]` contains the unique elements in the first `idx` positions.



```

def removeDuplicates(arr):
    n = len(arr)
    if n <= 1:
        return n

```

```

# Start from the second element
idx = 1

for i in range(1, n):
    if arr[i] != arr[i - 1]:
        arr[idx] = arr[i]
        idx += 1

return idx

if __name__ == "__main__":
    arr = [1, 2, 2, 3, 4, 4, 4, 5, 5]
    newSize = removeDuplicates(arr)

    for i in range(newSize):
        print(arr[i], end=" ")

```

Output

```
1 2 3 4 5
```

Comment

Sahil C

14 .

Rearrange Array by Sign

Last Updated : 3 Apr, 2026

- Given an array **arr[]** of size n, the task is to rearrange it in **alternate** positive and negative manner without changing the relative order of positive and negative numbers. In case of extra positive/negative numbers, they appear at the end of the array.

Note: The rearranged array should start with a positive number and 0 (zero) should be considered as a positive number.

Examples:

Input: **arr[]** = [1, 2, 3, -4, -1, 4]

Output: **arr[]** = [1, -4, 2, -1, 3, 4]

Input: **arr[]** = [-5, -2, 5, 2, 4, 7, 1, 8, 0, -8]

Output: **arr[]** = [5, -5, 2, -2, 4, -8, 7, 1, 8, 0]

Try It Yourself

The idea is to separate the numbers into **positive** and **negative arrays**. Then, alternately place numbers from each array back into the original array. Also place any remaining positive or negative numbers at the end.

Working of Code With Example :

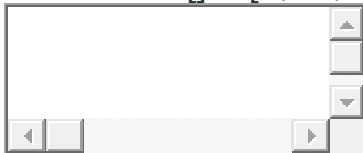
1. Let us consider arr = [1, 2, 3, -4, -1, 4]
2. First we create two vectors to store positive and negative numbers of the given array

pos[] = [1, 2, 3, 4], neg[] = [-4, -1]

4. Now we traverse both pos[] and neg[] and fill elements back into arr[] at alternate indexes

- i = 0, Since i is even, we pick from pos, arr = [1, _, _, _, _, _]
 - i = 1, Since i is odd, we pick from neg, arr = [1, -4, _, _, _, _]
 - i = 2, Since i is even, we pick from pos, arr = [1, -4, 2, _, _, _]
 - i = 3, Since i is odd, we pick from neg, arr = [1, -4, 2, -1, _, _]
5. One List Exhausted, neg[] in this case, so we put remaining pos[] elements

- Remaining pos[] = [3, 4]
- Final arr[] = [1, -4, 2, -1, 3, 4]



```
# Python program to rearrange positive and negative numbers  
# alternately using Two pointers
```

```
# Function to rearrange positive and negative integers  
# in alternate fashion
```

```
def rearrange(arr):  
    pos = []  
    neg = []  
  
    # Separate positive and negative numbers  
    for num in arr:  
        if num >= 0:  
            pos.append(num)
```

```

        else:
            neg.append(num)

posIdx = 0
negIdx = 0
i = 0

# Place positive and negative numbers alternately
# in the original array
while posIdx < len(pos) and negIdx < len(neg):
    if i % 2 == 0:
        arr[i] = pos[posIdx]
        posIdx += 1
    else:
        arr[i] = neg[negIdx]
        negIdx += 1
    i += 1

# Append remaining positive numbers (if any)
while posIdx < len(pos):
    arr[i] = pos[posIdx]
    posIdx += 1
    i += 1

# Append remaining negative numbers (if any)
while negIdx < len(neg):
    arr[i] = neg[negIdx]
    negIdx += 1
    i += 1

if __name__ == "__main__":
    arr = [1, 2, 3, -4, -1, 4]
    rearrange(arr)
    print(' '.join(map(str, arr)))

```

Output

```
1 -4 2 -1 3 4
```

Time Complexity : $O(n)$

Auxiliary Space : $O(n)$

Leaders in an array

Last Updated : 22 Jan, 2026

- Given an array `arr[]` of size `n`, the task is to find all the Leaders in the array. An element is a Leader if it is greater than or equal to all the elements to its right side.

Note: The rightmost element is always a leader.

Examples:

Input: `arr[] = [16, 17, 4, 3, 5, 2]`

Output: `[17 5 2]`

Explanation: 17 is greater than all the elements to its right i.e., [4, 3, 5, 2], therefore 17 is a leader. 5 is greater than all the elements to its right i.e., [2], therefore 5 is a leader. 2 has no element to its right, therefore 2 is a leader.

Input: `arr[] = [1, 2, 3, 4, 5, 2]`

Output: `[5 2]`

Explanation: 5 is greater than all the elements to its right i.e., [2], therefore 5 is a leader. 2 has no element to its right, therefore 2 is a leader.

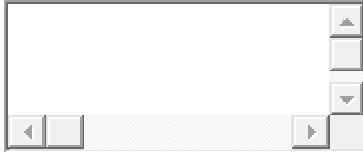
[Try It Yourself](#)

Table of Content

- [\[Naive Approach\] Using Nested Loops - \$O\(n^2\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Expected Approach\] Using Suffix Maximum - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)

[Naive Approach] Using Nested Loops - $O(n^2)$ Time and $O(1)$ Space:

Use two loops. The outer loop runs from 0 to size - 1 and one by one pick all elements from left to right. The inner loop compares the picked element to all the elements on its right side. If the picked element is greater than all the elements to its right side, then the picked element is the leader.



```
# Function to find the leaders in an array
def leaders(arr):
    result = []
    n = len(arr)

    for i in range(n):

        # Check elements to the right
        for j in range(i + 1, n):

            # If a larger element is found
            if arr[i] < arr[j]:
                break
        else:
            # If no larger element was found
            result.append(arr[i])

    return result

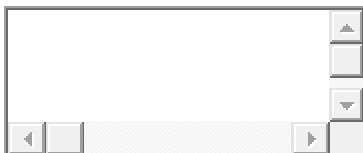
if __name__ == "__main__":
    arr = [16, 17, 4, 3, 5, 2]
    result = leaders(arr)
    print(" ".join(map(str, result)))
```

Output

```
17 5 2
```

[Expected Approach] Using Suffix Maximum - O(n) Time and O(1) Space:

*The idea is to scan all the elements from **right to left** in an array and keep track of the maximum till now. When the maximum changes its value, add it to the result. Finally reverse the result*



```
# Function to find the leaders in an array
def leaders(arr):
    result = []
    n = len(arr)
```

```

# Start with the rightmost element
maxRight = arr[-1]

# Rightmost element is always a leader
result.append(maxRight)

# Traverse the array from right to left
for i in range(n - 2, -1, -1):
    if arr[i] >= maxRight:
        maxRight = arr[i]
        result.append(maxRight)

# Reverse the result list to maintain
# original order
result.reverse()

return result

if __name__ == "__main__":
    arr = [16, 17, 4, 3, 5, 2]
    result = leaders(arr)
    print(" ".join(map(str, result)))

```

Output

```
17 5 2
```

Illustration:

$arr[] = \{16, 17, 4, 3, 5, 2\}$

Initially : $maxRight = 2, res[] = \{ 2 \}$

- *$i = 4, maxRight = 5, res[] = \{ 2, 5 \}$*
- *$i = 3, maxRight = 5, res[] = \{ 2, 5 \}$*
- *$i = 2, maxRight = 5, res[] = \{ 2, 5 \}$*
- *$i = 1, maxRight = 17, res[] = \{ 2, 5, 17 \}$*
- *$i = 0, maxRight = 17, res[] = \{ 2, 5, 17 \}$*

Reverse $res[] = \{17, 5, 2\}$ and return

16.

Missing and Repeating in an Array

Last Updated : 9 Feb, 2026

- Given an unsorted array `arr[]` of size n , containing elements from the range 1 to n , it is known that one number in this range is missing, and another number occurs twice in the array, find both the **duplicate** number and the **missing** number.

Examples:

Input: `arr[] = [3, 1, 3]`

Output: `[3, 2]`

Explanation: *3 is occurs twice and 2 is missing.*

Input: `arr[] = [4, 3, 6, 2, 1, 1]`

Output: `[1, 5]`

Explanation: *1 is occurs twice and 5 is missing.*

[Try It Yourself](#)

Table of Content

- [\[Approach 1\] Using Visited Array - \$O\(n\)\$ Time and \$O\(n\)\$ Space](#)
- [\[Approach 2\] Using Array Marking - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Approach 3\] Making Two Math Equations - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Approach 4\] Using XOR - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)

[Approach 1] Using Visited Array - $O(n)$ Time and $O(n)$ Space

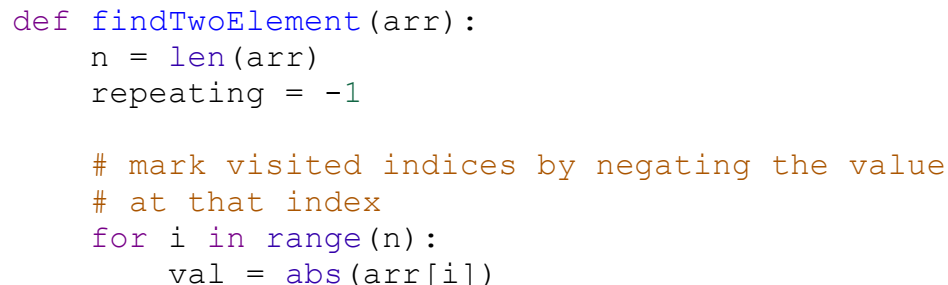
The idea is to use a frequency array to keep track of how many times each number appears in the original array. Since we know the numbers should range from 1 to n with each appearing exactly once, any number appearing twice is our repeating number, and any number with zero frequency is our missing number.



```
def findTwoElement(arr):  
    n = len(arr)  
  
    # frequency array to count occurrences
```

Output

The main idea is to use the input array itself for tracking: it negates the value at the index corresponding to each element to mark it as visited. If it encounters a value that has already been negated, it identifies that number as the repeating one. In a second pass, it finds the index that remains positive, which corresponds to the missing number.



```

        # if value at index val - 1 is already negative,
        # val is repeating
        if arr[val - 1] > 0:
            arr[val - 1] = -arr[val - 1]
        else:
            # Already visited → repeating element
            repeating = val

missing = -1

# the index with a positive value corresponds
# to the missing number
for i in range(n):
    if arr[i] > 0:
        missing = i + 1
        break

return [repeating, missing]

if __name__ == "__main__":
    arr = [3, 1, 3]
    ans = findTwoElement(arr)
    print(ans[0], ans[1])

```

Output

```
3 2
```

[Approach 3] Making Two Math Equations - $O(n)$ Time and $O(1)$ Space

The idea is to use mathematical equations based on the sum and sum of squares of numbers from 1 to n . The difference between expected and actual sums will give us one equation, and the difference between expected and actual sum of squares will give us another equation. Solving these equations yields our missing and repeating numbers.

Illustration:

Note: This method can cause arithmetic overflow as we calculate the sum of squares (or product) and sum of all array elements.



```
def findTwoElement(arr):
    n = len(arr)

    # Expected sum and sum of squares for numbers from 1 to n
    s = (n * (n + 1)) // 2
    ssq = (n * (n + 1) * (2 * n + 1)) // 6

    missing = 0
    repeating = 0

    # Subtract actual sum and sum of squares from expected
    values
    for num in arr:
        s -= num
        ssq -= num * num

    # Let s = x - y and ssq = x^2 - y^2 = (x - y) (x + y)
    # => x = (s + ssq // s) // 2, y = x - s
    missing = (s + ssq // s) // 2
```

```

    repeating = missing - s

    return [repeating, missing]

if __name__ == "__main__":
    arr = [3, 1, 3]
    ans = findTwoElement(arr)

    print(ans[0], ans[1])

```

Output

```
3 2
```

An Alternate way to make two equations:

- Let x be the missing and y be the repeating element.
- Get the sum of all numbers using formula $S = n(n+1)/2 - x + y$
- Get product of all numbers using formula $P = 1*2*3*...*n * y / x$
- The above two steps give us two equations, we can solve the equations and get the values of x and y.

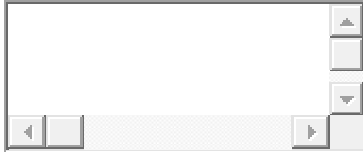
[Approach 4] Using XOR - O(n) Time and O(1) Space

The idea is to use XOR operations to isolate the missing and repeating numbers. By XORing all array elements with numbers 1 to n, we get the XOR of our missing and repeating numbers. Then, using a set bit in this XOR result, we can divide all numbers into two groups, which helps us separate the missing and repeating numbers.

Refer to [Find the two numbers with odd occurrences in an unsorted array](#) to understand how groups will be created.

Step by step approach:

- XOR all array elements and numbers from 1 to n to get XOR of missing and repeating numbers.
- Find the rightmost set bit in this XOR result using `xorVal & ~(xorVal-1)`.
- Use this set bit to divide array elements and numbers 1 to n into two groups.
- XOR elements of first group to get x and second group to get y.
- Count occurrences of x in original array to determine which is missing and which is repeating.
- If x appears in array, x is repeating and y is missing; otherwise vice versa.
- Return both the repeating and missing numbers.



```
def findTwoElement(arr):
    n = len(arr)
    xorVal = 0

    # get the xor of all array elements
    # And numbers from 1 to n
    for i in range(n):
        xorVal ^= arr[i]
        xorVal ^= (i + 1) # 1 to n numbers

    # get the rightmost set bit in xorVal
    setBitIndex = xorVal & ~(xorVal - 1)

    x, y = 0, 0

    # now divide elements into two sets
    # by comparing rightmost set bit
    for i in range(n):

        # decide whether arr[i] is in first set
        # or second
        if arr[i] & setBitIndex:
            x ^= arr[i]
        else:
            y ^= arr[i]

        # decide whether (i+1) is in first set
        # or second
        if (i + 1) & setBitIndex:
            x ^= (i + 1)
        else:
            y ^= (i + 1)

    # x and y are the repeating and missing values.
    # to know which one is what, traverse the array
    xCnt = sum(1 for num in arr if num == x)

    if xCnt == 0:
        missing, repeating = x, y
    else:
        missing, repeating = y, x

    return [repeating, missing]
```

```
if __name__ == "__main__":  
    arr = [3, 1, 3]  
    ans = findTwoElement(arr)  
  
    print(ans[0], ans[1])
```

Output

```
3 2
```

17.

Missing ranges of numbers

Last Updated : 2 Jan, 2025

- You have an inclusive interval [**lower**, **upper**] and a sorted array of unique integers **arr[]**, all of which lie within this interval. A number **x** is considered missing if **x** is in the range [**lower**, **upper**] but not present in **arr**. Your task is to return the smallest set of sorted ranges that includes all missing numbers, ensuring no element from **arr** is within any range, and every missing number is covered exactly once.

Examples

Input: **arr[]** = [14, 15, 20, 30, 31, 45], **lower** = 10, **upper** = 50

Output: [[10, 13], [16, 19], [21, 29], [32, 44], [46, 50]]

Explanation: These ranges represent all missing numbers between 10 and 50 not present in the array

Input: **arr[]** = [-48, -10, -6, -4, 0, 4, 17], **lower** = -54, **upper** = 17

Output: [[-54, -49], [-47, -11], [-9, -7], [-5, -5], [-3, -1], [1, 3], [5, 16]]

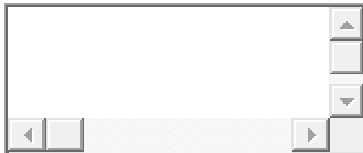
Explanation: These ranges represent all missing numbers between -54 and 17 not present in the array.

[Try It Yourself](#)

Using Linear Scan - $O(n)$ Time and $O(1)$ Space

Since `arr[]` is sorted, we can efficiently identify gaps between consecutive elements. By iterating through `arr[]`, we check the difference between adjacent numbers:

- If the difference is greater than one, the numbers in between form a missing range.
- Additionally, we handle edge cases where the missing range starts before the first element or ends after the last element of `arr[]`.



```
# Python program to calculate missing ranges in an array
def missingRanges(arr, lower, upper):
    n = len(arr)
    res = []

    # Check for missing range before the first element
    if lower < arr[0]:
        res.append([lower, arr[0] - 1])

    # Check for missing ranges between consecutive elements
    for i in range(n - 1):
        if arr[i + 1] - arr[i] > 1:
            res.append([arr[i] + 1, arr[i + 1] - 1])

    # Check for missing range after the last element
    if upper > arr[-1]:
        res.append([arr[-1] + 1, upper])

    return res

if __name__ == "__main__":
    lower = 10
    upper = 50
    arr = [14, 15, 20, 30, 31, 45]
    res = missingRanges(arr, lower, upper)
    for v in res:
        print(v[0], v[1])
```

Output

```
10 13
16 19
```

21 29

32 44

46 50

Comment

18.

Sum of all Subarrays

Last Updated : 22 Jul, 2025

- Given an integer array **arr[]**, compute the sum of all possible sub-arrays of the array. A sub-array is a contiguous part of the array.
Examples:
Input: *arr[] = [1, 4, 5, 3, 2]*
Output: 116
Explanation: Sum of all possible subarrays of the array [1, 4, 5, 3, 2] is 116.
Input: *arr[] = [1, 2, 3, 4]*
Output: 50
Explanation: Sum of all possible subarrays of the array [1, 2, 3, 4] is 50.

[Try It Yourself](#)

Table of Content

- [\[Naive Approach\] Using Nested Loop - \$O\(n^2\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Expected Approach\] Element Contribution Method - \$O\(n\)\$ Time and \$O\(1\)\$ Space](#)

[Naive Approach] Using Nested Loop - $O(n^2)$ Time and $O(1)$ Space

The main idea of this approach is to iterate over all possible subarrays of the given array using two nested loops. The outer loop selects the starting index

of the subarray, while the inner loop extends the subarray to include all possible ending points. During this process, a running sum (temp) is maintained for each subarray and added to the total result.



```
def subarraySum(arr):
    n = len(arr)
    result = 0

    # pick starting point
    for i in range(n):
        temp = 0

        # pick ending point
        for j in range(i, n):

            # sum subarray between current starting
            # and ending points
            temp += arr[j]
            result += temp

    return result

if __name__ == "__main__":
    arr = [1, 4, 5, 3, 2]
    print(subarraySum(arr))
```

Output

116

[Expected Approach] Element Contribution Method - $O(n)$ Time and $O(1)$ Space

If we take a close look then we observe a pattern.

Let's take an example: $arr[] = [1, 4, 5, 3, 2]$

All subarrays : $[1], [1, 4], [1, 4, 5], [1, 4, 5, 3], [1, 4, 5, 3, 2], [4], [4, 5], [4, 5, 3], [4, 5, 3, 2], [5], [5, 3], [5, 3, 2], [3], [3, 2], [2]$

Third element $arr[2]$ appears 3 time when subarray start with $arr[0]$: $[1, 4, 5]$, $[1, 4, 5, 3]$, $[1, 4, 5, 3, 2]$

Third element $arr[2]$ appears 3 time when subarray start with $arr[1]$: $[4, 5]$, $[4, 5, 3]$, $[4, 5, 3, 2]$

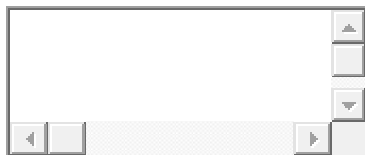
Third element $arr[2]$ appears 3 time when subarray start with $arr[2]$: $[5]$, $[5, 3]$, $[5, 3, 2]$

So, We can clearly see that, For any element $arr[i]$ in an array of size n , it appears in exactly $(i + 1) * (n - i)$ subarrays.

For $arr[] = [1, 2, 3]$, sum of subarrays is: $arr[0] * (0 + 1) * (3 - 0) + arr[1] * (1 + 1) * (3 - 1) + arr[2] * (2 + 1) * (3 - 2) = 1*3*3 + 2*2*2 + 3*3*1 = 20$

Step by step Implementation:

- Declare an integer answer equal to zero
- Run a for loop for i from $[0, n-1]$: Add $arr[i] * (i+1) * (n-i)$ into the answer at each iteration
- Return answer



```
def subarraySum(arr):
```

```
    n = len(arr)
```

```
result = 0

# Computing sum of subarrays using the formula
for i in range(n):
    result += arr[i] * (i + 1) * (n - i)

# Return the sum of all subarrays
return result

if __name__ == "__main__":
    arr = [1, 4, 5, 3, 2]
    print(subarraySum(arr))
```

Output

116